

SIGNAL

Доказуемая инфраструктура,
а не просто зелёные Pod.

Nginx в Kubernetes с Gateway API, Prometheus и Fluentd → Loki. Один контрольный запрос проверяется на входе, в метриках и в собранных логах; сбой canary превращён в воспроизводимый эксперимент.

Статус проверки — существенное ограничение

55 локальных тестов пройдены: Debian 13, Nginx 1.26.3 и настоящий TLS handshake. Выбранные контейнеры, GitHub CI и Ubuntu 24.04 / kubeadm E2E в среде подготовки НЕ запускались. До сдачи требуется make acceptance и обновление паспорта по фактическому отчёту.

01 / Архитектура и состав

Слой	Выбранная реализация
Целевая среда	Ubuntu 24.04 amd64; один выделенный узел; kubeadm 1.35.9 + containerd из Ubuntu.
Сеть и вход	Cilium 1.20.2; Envoy Gateway v1.9.2; Gateway API v1.6.1; NodePort 30080 / 30443.
Веб-приложение	Nginx 1.30.5-alpine: stable ×2, canary ×1; Hello World!; JSON access + stderr error logs.
Наблюдаемость	Prometheus 3.13.3; Loki и образ Fluentd/Loki plugin 3.7.8 (база Fluentd 1.19); Grafana 13.1.0.
Автоматизация	Bash + Make + Python stdlib; Helm 3.20.2 для Cilium/Envoy; JSON manifests для остальных ресурсов.

Путь данных

Эксперт → Envoy Gateway API → Service → Nginx
Nginx stdout/stderr → Fluentd + buffer → Loki → Grafana
Nginx / Envoy / узел / Blackbox → Prometheus → Grafana
verify.py → HTTP + PromQL + LogQL → report.json / report.html

Prometheus/Loki/Grafana/Alertmanager имеют local PV. Админ-интерфейсы доступны только через localhost port-forward. Облако и коммерческие сервисы не обязательны.

02 / Обязательный функционал

В таблице — что реализовано в коде, почему это выбрано и чем проверяется. Успех runtime-команд должен быть получен на реальном стенде; наличие конфигураций само по себе не является подтверждением.

Требование	Реализация и обоснование	Как проверить
Kubernetes	kubeadm bootstrap на отдельной Ubuntu 24.04 VM; фиксирован Kubernetes 1.35.9. Это приоритетный способ в кейсе, без vendor-specific облака.	Bootstrap из README; затем make acceptance
Веб-приложение	Готовый Nginx вместо разработки нового приложения; точное тело Hello World!, версии через X-Release, probes и 2 stable Pod.	curl через NodePort с Host: signal.local; HTTP 200 и точное тело ответа.
Gateway API	GatewayClass, Gateway, HTTPRoute → Service; EnvoyProху управляет NodePort. Нет подмены Ingress или прямого доступа к приложению.	make verify: оба listener, текущий observedGeneration, HTTP/TLS и проверяемый ответ.
Prometheus	Deployment + PVC; 6 scrape jobs: nginx, envoy, node, loki, prometheus, gateway-probe. Targets и реальные query, а не только dashboard.	make verify: непустые finite metrics, свежий scrape, up и рост счётчика после 40 запросов.
Fluentd / логи	DaemonSet читает CRI-логи только Nginx; persistent positions и buffer; Loki хранит stdout access и stderr error.	make verify: уникальный proof_id найден через Loki; /missing/ создаёт 404 и собранный error-log.
Повторный запуск	Makefile, deterministic renderer, server-side dry-run, ожидание rollout; существующие Secrets не заменяются.	make acceptance: deploy дважды, verify до/после, сравнение UID Pod/PVC и публичных сертификатов.
Версии и инструкция	README с архитектурой, средой, командами и ограничениями; image-lock resolver, chart vendor/SHA-256.	make lock && make vendor && make render; затем свежая приёмка этого commit.

Что именно считается доказательством

Отчёт сохраняет реальный ответ, conditions, PromQL/LogQL результаты, версии, imageID, commit, ОС и длительности. Пустой вектор метрик, устаревший статус маршрута, ненайденный лог или пропущенная сетевая проверка дают FAIL, а не успешный скриншот.

Nginx stub_status учитывает и служебные запросы. Рост его счётчика не выдаётся за точную атрибуцию конкретному пользователю; конкретный запрос подтверждается отдельным proof_id в Loki.

03 / Дополнения и контроль отказов

Дополнения усиливают основные критерии: их можно воспроизвести теми же командами, а не только прочитать о них в паспорте.

Улучшение	Реализация / смысл	Проверка
Маршрутизация и TLS	Hostname signal.local; /stable, /canary; header X-Release; несколько backend; локальный CA и настоящий SNI.	make verify; TLS с --cacert и --resolve, без -k.
Traffic splitting	HTTPRoute backend weights; 400 запросов при 90/10 с широким допуском на случайность.	make demo; в отчёте фактические counts, не обещание ровно 10%.
Canary gate и откат	10 → 25 → 50 → 100%; по 30 запросов именно к canary; ответ/версия, ≤1% ошибок, p95 ≤500 ms, минимум samples и мониторинг.	make canary; make demo намеренно возвращает 503 при живом /healthz и проверяет отклонение/откат.
Исполняемая изоляция	Cilium NetworkPolicy: к Nginx только свой Gateway и Prometheus. Restricted Job из другого namespace сначала проходит через Gateway, затем не проходит напрямую.	make verify: positive + negative controls, а не лишь наличие policy.
Восстановление и хранение	Probes/PDB/graceful stop; удаление одного stable Pod; persistent local PV и ограниченные retention/buffer.	make demo: реально измеренные ошибки/время; make acceptance: сохранение идентичности PVC.
Dashboard и CI	Grafana с Prometheus/Loki, blackbox, CPU/RAM, latency и local alerts; hosted Ubuntu workflow static → kind → demo → evidence.	make ui; .github/workflows/ci.yml. Успешный CI-прогон в поставке не заявлен.

Безопасность по умолчанию

Nginx: non-root, read-only rootfs, drop ALL, seccomp, requests/limits. Prometheus читает только Pods в разрешённых namespaces; нет доступа к Secrets. Пароли и ключи генерируются локально, исключены из Git. Privileged namespace-исключения для host logs и host metrics отделены от приложения; Fluentd остаётся root-процессом с чувствительным доступом к логам.

Границы эксперимента

500 ms — демонстрационный порог, не измеренный SLA. CLI canary не является постоянным rollout-controller. При отказе API/SIGKILL откат не гарантирован; восстановление baseline: make deploy && make verify. Alertmanager не отправляет внешние уведомления.

04 / Ревью и масштабирование

Главная техническая особенность

Единый протокол связывает настоящий запрос через Gateway с доступной телеметрией и найденным логом. Отрицательные контроли проверяют сам механизм проверки: пустые данные, чужой hostname, запрещённый обход и ошибочный сапагу не маскируются зелёными Pod. Это применение известных практик к кейсу, а не заявление об абсолютной уникальности.

Самое сложное архитектурное решение

Выбор между максимально богатым стеком и системой, которую эксперт воспроизведёт на одной VM. Приняты kubeadm + NodePort + local PV и небольшой проверяемый renderer, а не облачный LB, service mesh и распределённое хранилище до подтверждения основы. Цена решения — отсутствие node-level HA и необходимость отдельной финальной приёмки.

Потенциальное развитие с учётом телекома

Направление	Что понадобится
Edge / несколько площадок	Отдельные кластеры, site labels, локальная автономность при разрыве WAN, защищённый канал сбора; VM, сеть и PKI.
Хранение и disaster recovery	Несколько узлов, CSI, S3-совместимое хранилище, off-node backup и проверка restore; дополнительные диски/узлы.
Клиентская изоляция	Tenant-aware authorization, quotas, mTLS и очистка PII; модель арендаторов и правила доступа/хранения.
SLO и supply chain	Долгие окна наблюдения, baseline comparison, burn-rate alerts; подписи образов, SBOM/CVE, полный offline bundle и процесс обновлений.

Ограничения текущей реализации

Один узел и local disk — единые точки отказа. Нет exactly-once, автоматического TLS renew, multiline/CRI partial assembly, end-to-end mTLS и гарантии нулевой потери логов. Access-лог минимизирован, но штатный error-лог может содержать IP/URI; стенд предназначен для синтетических данных. Загрузка пакетов/образов требует публичных upstream-источников; digest lock и vendor ещё должны быть получены при подготовке финального стенда.

Материалы и порядок финализации

README и docs/ содержат команды, runbook, критерии и сценарий защиты. После успешного Ubuntu acceptance обновить этот паспорт фактическими результатами, опубликовать проверенный main и выполнить make package с настоящими URL/фамилией. Сборщик проверяет отчёт, main commit, наличие lock/vendor, чистоту worktree и лимит 4 страниц PDF; фиктивная ссылка не создаётся.

Основание: предоставленный кейс, разделы «Критерии оценки» и «Требования к сдаче решения». Технические первичные источники и версии — docs/SOURCES.md. В кейсе срок указан как 4 октября 23:59 без года/часового пояса; уточнить на странице конкурса. Изменения main после официального cutoff запрещены.